

```
import os
import shutil

dataset_paths = [
    '/content/Dataset1/',
    '/content/Dataset2/',
    '/content/Dataset3/',
    '/content/Dataset4/',
    '/content/Dataset5/',
    '/content/Dataset6/',
    '/content/Dataset7/',
    '/content/Dataset8/',
]

combined_dataset_path = '/content/CombinedDataset/'

os.makedirs(os.path.join(combined_dataset_path, 'images'), exist_ok=True)
os.makedirs(os.path.join(combined_dataset_path, 'labels'), exist_ok=True)

for path in dataset_paths:
    images_path = os.path.join(path, 'images')
    labels_path = os.path.join(path, 'labels')

    for file_name in os.listdir(images_path):
        full_file_name = os.path.join(images_path, file_name)
        if os.path.isfile(full_file_name):
            shutil.copy(full_file_name, os.path.join(combined_dataset_path, 'images', file_name))

    for file_name in os.listdir(labels_path):
        full_file_name = os.path.join(labels_path, file_name)
        if os.path.isfile(full_file_name):
            shutil.copy(full_file_name, os.path.join(combined_dataset_path, 'labels', file_name))
```

```
!git clone https://github.com/ultralytics/yolov5 # clone
%cd yolov5
!pip install -qr requirements.txt # install

import torch
import utils
display = utils.notebook_init() # checks
```

YOLOv5  v7.0-347-gd6112173 Python-3.10.12 torch-2.3.1+cu121 CPU
Setup complete  (2 CPUs, 12.7 GB RAM, 30.2/107.7 GB disk)

```
!pip install -U -q PyDrive

from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials
import zipfile

# Authenticate and create the PyDrive client.
auth.authenticate_user()
gauth = GoogleAuth()
gauth.credentials = GoogleCredentials.get_application_default()
drive = GoogleDrive(gauth)

def download_file(file_id, dest_path):
    downloaded = drive.CreateFile({'id': file_id})
    downloaded.GetContentFile(dest_path)
    print(f"Downloaded {dest_path}")

file_id = [REDACTED] # Replace with your actual file ID
dest_path = '/content/combined_dataset.zip' # Replace with your desired destination file name

download_file(file_id, dest_path)
```

WARNING:root:pydrive is deprecated and no longer maintained. We recommend that you migrate your projects to pydrive2, the maintained fork.
Downloaded /content/combined_dataset.zip

开始借助 AI 编写或生成代码。

```
!python export.py --weights /content/weights/best.pt --include tflite
```

```
tf_concat_2 (TFConcat)      (1, 40, 40, 256)      0      ['tf_conv_45[0][0]',
      'tf_conv_39[0][0]']

tfc3_6 (TFC3)              (1, 40, 40, 256)      295680  ['tf_concat_2[0][0]']

tf_conv_51 (TFConv)        (1, 20, 20, 256)      590080  ['tfc3_6[0][0]']

tf_concat_3 (TFConcat)     (1, 20, 20, 512)      0      ['tf_conv_51[0][0]',
      'tf_conv_33[0][0]']

tfc3_7 (TFC3)              (1, 20, 20, 512)      1181184 ['tf_concat_3[0][0]']

tf_detect (TFDetect)       ((1, 25200, 11),      29667  ['tfc3_5[0][0]',
      )                               'tfc3_6[0][0]',
                                    'tfc3_7[0][0]']

=====
Total params: 7026307 (26.80 MB)
Trainable params: 0 (0.00 Byte)
Non-trainable params: 7026307 (26.80 MB)

TensorFlow SavedModel: export success 14.1s, saved as /content/weights/best_saved_model (27.1 MB)

TensorFlow Lite: starting export with tensorflow 2.15.0...
Summary on the non-converted ops:

-----
* Accepted dialects: tfl, builtin, func
* Non-Converted Ops: 146, Total Ops 535, % non-converted = 27.29 %
* 146 ARITH ops

- arith.constant: 146 occurrences  (f16: 131, i32: 15)

(f32: 10)
(f32: 17)
(f32: 60)
(f32: 131)
(f32: 66)
(f32: 3)
(f32: 75)
(f32: 7)
(f32: 6)
(f32: 2)
(f32: 9)
TensorFlow Lite: export success 79.8s, saved as /content/weights/best-fp16.tflite (13.5 MB)

Export complete (95.5s)
Results saved to /content/weights
Detect:      python detect.py --weights /content/weights/best-fp16.tflite
Validate:    python val.py --weights /content/weights/best-fp16.tflite
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', '/content/weights/best-fp16.tflite')
Visualize:   https://netron.app
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!unzip -q /content/combined_dataset.zip -d.../
```

```
import os
from collections import defaultdict

# Define paths
labels_dir = '/labels' # Path to your labels directory

def identify_images_containing_classes(labels_dir):
    class_to_images = defaultdict(set)
    image_to_classes = defaultdict(set)

    for filename in os.listdir(labels_dir):
        if filename.endswith('.txt'):
            label_file_path = os.path.join(labels_dir, filename)
            with open(label_file_path, 'r') as file:
                for line in file:
                    class_id = int(line.split()[0])
                    image_to_classes[filename].add(class_id)
                    class_to_images[class_id].add(filename)

    return class_to_images, image_to_classes

class_to_images, image_to_classes = identify_images_containing_classes(labels_dir)
print("Number of images per class:", {k: len(v) for k, v in class_to_images.items()})
```

Number of images per class: {4: 4748, 3: 3173, 0: 5925, 5: 3423, 2: 3174, 1: 3262}

```
!python train.py --img 640 --epochs 30 --data /content/data.yaml --weights yolov5s.pt
```

2024-07-18 18:32:07.432862: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261] Unable to register cuDNN factory: Attempti
2024-07-18 18:32:07.432931: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607] Unable to register cuFFT factory: Attemptin
2024-07-18 18:32:07.544915: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515] Unable to register cuBLAS factory: Attemp
train: weights=yolov5s.pt, cfg=, data=/content/data.yaml, hyp=data/hyps/hyp.scratch-low.yaml, epochs=30, batch_size=16, imgsz=640, re
github: up to date with <https://github.com/ultralytics/yolov5> 
YOLOv5  v7.0-345-g8003649c Python-3.10.12 torch-2.3.1+cu121 CPU

hyperparameters: lr0=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1
Comet: run 'pip install comet_ml' to automatically track and visualize YOLOv5  runs in Comet
TensorBoard: Start with 'tensorboard --logdir runs/train', view at <http://localhost:6006/>
Overriding model.yaml nc=80 with nc=6

	from	n	params	module	arguments
0		-1 1	3520	models.common.Conv	[3, 32, 6, 2, 2]
1		-1 1	18560	models.common.Conv	[32, 64, 3, 2]
2		-1 1	18816	models.common.C3	[64, 64, 1]
3		-1 1	73984	models.common.Conv	[64, 128, 3, 2]
4		-1 2	115712	models.common.C3	[128, 128, 2]
5		-1 1	295424	models.common.Conv	[128, 256, 3, 2]
6		-1 3	625152	models.common.C3	[256, 256, 3]
7		-1 1	1180672	models.common.Conv	[256, 512, 3, 2]
8		-1 1	1182720	models.common.C3	[512, 512, 1]
9		-1 1	656896	models.common.SPPF	[512, 512, 5]
10		-1 1	131584	models.common.Conv	[512, 256, 1, 1]
11		-1 1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	models.common.Concat	[1]
13		-1 1	361984	models.common.C3	[512, 256, 1, False]
14		-1 1	33024	models.common.Conv	[256, 128, 1, 1]
15		-1 1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
16	[-1, 4]	1	0	models.common.Concat	[1]
17		-1 1	90880	models.common.C3	[256, 128, 1, False]
18		-1 1	147712	models.common.Conv	[128, 128, 3, 2]
19	[-1, 14]	1	0	models.common.Concat	[1]
20		-1 1	296448	models.common.C3	[256, 256, 1, False]
21		-1 1	590336	models.common.Conv	[256, 256, 3, 2]
22	[-1, 10]	1	0	models.common.Concat	[1]
23		-1 1	1182720	models.common.C3	[512, 512, 1, False]
24	[17, 20, 23]	1	29667	models.yolo.Detect	[6, [[10, 13, 16, 30, 33, 23], [30, 61, 62, 45, 59, 119],

Model summary: 214 layers, 7035811 parameters, 7035811 gradients, 16.0 GFLOPs

Transferred 343/349 items from yolov5s.pt
optimizer: SGD(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01), CLAHE(p=0.01, clip_limit=(1,
/usr/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithread
self.pid = os.fork()
train: Scanning /content/dataset_split/train/labels... 16410 images, 0 backgrounds, 0 corrupt: 100% 16410/16410 [00:14<00:00, 1157.34
train: New cache created: /content/dataset_split/train/labels.cache
val: Scanning /content/dataset_split/valid/labels... 4688 images, 0 backgrounds, 0 corrupt: 100% 4688/4688 [00:07<00:00, 612.78it/s]
val: New cache created: /content/dataset_split/valid/labels.cache

AutoAnchor: 2.80 anchors/target, 1.000 Best Possible Recall (BPR). Current anchors are a good fit to dataset 
Plotting labels to runs/train/exp2/labels.jpg..
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to runs/train/exp2
Starting training for 30 epochs...

```
!python export.py --weights /content/yolov5/runs/train/exp/weights/best.pt --include tflite

export: data=data/coco128.yaml, weights=['/content/yolov5/runs/train/exp/weights/best.pt'], imgsz=[640, 640], batch_size=1, device=cpu
YOLOv5 v7.0-342-g12be4996 Python-3.10.12 torch-2.3.0+cu121 CPU

Fusing layers...
Model summary: 157 layers, 7026307 parameters, 0 gradients, 15.8 GFLOPs

PyTorch: starting from /content/yolov5/runs/train/exp/weights/best.pt with output shape (1, 25200, 11) (13.8 MB)
2024-07-16 08:08:59.032191: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261] Unable to register cuDNN factory: Attempti
2024-07-16 08:08:59.032243: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607] Unable to register cuFFT factory: Attemptin
2024-07-16 08:08:59.033544: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515] Unable to register cuBLAS factory: Attempt

TensorFlow SavedModel: starting export with tensorflow 2.15.0...
WARNING ⚠ using Tensorflow 2.15.0 > 2.13.1 might cause issue when exporting the model to tflite https://github.com/ultralytics/yolov5/

from n      params module                                arguments
2024-07-16 08:09:01.505951: E external/local_xla/xla/stream_executor/cuda/cuda_driver.cc:274] failed call to cuInit: CUDA_ERROR_NO_DEV
0          -1 1       3520 models.common.Conv          [3, 32, 6, 2, 2]
1          -1 1      18560 models.common.Conv          [32, 64, 3, 2]
2          -1 1      18816 models.common.C3           [64, 64, 1]
3          -1 1      73984 models.common.Conv          [64, 128, 3, 2]
4          -1 1     115712 models.common.C3           [128, 128, 2]
5          -1 1     295424 models.common.Conv          [128, 256, 3, 2]
6          -1 1     625152 models.common.C3           [256, 256, 3]
7          -1 1    1180672 models.common.Conv          [256, 512, 3, 2]
8          -1 1    1182720 models.common.C3           [512, 512, 1]
9          -1 1     656896 models.common.SPPF         [512, 512, 5]
10         -1 1     131584 models.common.Conv          [512, 256, 1, 1]
11         -1 1         0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12        [-1, 6] 1         0 models.common.Concat          [1]
13         -1 1     361984 models.common.C3           [512, 256, 1, False]
14         -1 1     33024  models.common.Conv          [256, 128, 1, 1]
15         -1 1         0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
16        [-1, 4] 1         0 models.common.Concat          [1]
17         -1 1     90880  models.common.C3           [256, 128, 1, False]
18         -1 1     147712 models.common.Conv          [128, 128, 3, 2]
19        [-1, 14] 1         0 models.common.Concat          [1]
20         -1 1     296448 models.common.C3           [256, 256, 1, False]
21         -1 1     590336 models.common.Conv          [256, 256, 3, 2]
22        [-1, 10] 1         0 models.common.Concat          [1]
23         -1 1    1182720 models.common.C3           [512, 512, 1, False]
24        [17, 20, 23] 1     29667  models.yolo.Detect          [6, [[10, 13, 16, 30, 33, 23], [30, 61, 62, 45, 59, 119],
Model: "model"
```

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(1, 640, 640, 3)]	0	[]
tf_conv (TFConv)	(1, 320, 320, 32)	3488	['input_1[0][0]']
tf_conv_1 (TFConv)	(1, 160, 160, 64)	18496	['tf_conv[0][0]']
tf_c3 (TFC3)	(1, 160, 160, 64)	18624	['tf_conv_1[0][0]']
tf_conv_7 (TFConv)	(1, 80, 80, 128)	73856	['tf_c3[0][0]']
tf_c3_1 (TFC3)	(1, 80, 80, 128)	115200	['tf_conv_7[0][0]']
tf_c3_2 (TFC3)	(1, 40, 40, 256)	230400	['tf_c3_1[0][0]']

```
!python detect.py --weights /content/weights/best.pt --img 640 --conf 0.25 --source /content/TestingImages

y
y
y
detect: weights=['/content/weights/best.pt'], source=/content/TestingImages, data=data/coco128.yaml, imgsz=[640, 640], conf_thres=0.25,
YOLOv5 v7.0-345-g8003649c Python-3.10.12 torch-2.3.1+cu121 CPU

Fusing layers...
Model summary: 157 layers, 7026307 parameters, 0 gradients, 15.8 GFLOPs
image 1/5 /content/TestingImages/Tensorflow_convert.jpeg: 512x640 1 fresh_apple, 412.1ms
image 2/5 /content/TestingImages/apples-royalty-free-image-164084111-1537885595.jpg: 448x640 11 fresh_apples, 296.9ms
image 3/5 /content/TestingImages/images (1).jpeg: 448x640 3 rotten_apples, 279.7ms
image 4/5 /content/TestingImages/images.jpeg: 416x640 5 fresh_apples, 256.8ms
image 5/5 /content/TestingImages/p07v2wjn.jpg: 384x640 14 fresh_apples, 369.8ms
Speed: 2.6ms pre-process, 323.0ms inference, 6.0ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs/detect/exp
```

```
import os
import shutil

#The Classes to keep
classes_to_keep = {0, 1, 3, 5, 6, 7}

# Define paths
source_images_dir = '/content/fruits-3/valid/images'
```

```

source_labels_dir = '/content/fruits-3/valid/labels'
dest_images_dir = '/content/images1'
dest_labels_dir = '/content/anno_images1'

os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep):
    valid_images = set() # To store the names of valid images

    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                class_id = int(line.split()[0])
                if class_id in classes_to_keep:
                    dst_file.write(line)

        # Check if the filtered label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            image_filename = filename.replace('.txt', '.jpg')
            valid_images.add(image_filename)

    # Copy valid images
    for image_filename in valid_images:
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define the classes you want to keep and their new labels
class_mapping = {0: 0, 1: 1, 3: 2, 5: 3, 6: 4, 7: 5}

# Define paths
source_images_dir = '/content/images1'
source_labels_dir = '/content/anno_images1'
dest_images_dir = '/content/images2'
dest_labels_dir = '/content/anno_images2'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                old_class_id = int(line.split()[0])
                if old_class_id in class_mapping:
                    new_class_id = class_mapping[old_class_id]
                    new_line = line.replace(str(old_class_id), str(new_class_id), 1)
                    dst_file.write(new_line)

        # Check if the filtered and relabeled label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            # Copy the corresponding image if the label file is not empty
            image_filename = filename.replace('.txt', '.jpg')
            source_image_path = os.path.join(source_images_dir, image_filename)

```

```

dest_image_path = os.path.join(dest_images_dir, image_filename)
shutil.copyfile(source_image_path, dest_image_path)

filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil
import random

# Define paths
source_images_dir = '/content/images'
source_labels_dir = '/content/labels'
base_dest_dir = '/content/dataset_split'

# Define the split ratios
train_ratio = 0.7
valid_ratio = 0.2
test_ratio = 0.1

# Ensure the ratios sum to 1 within a small tolerance
tolerance = 1e-6
assert abs(train_ratio + valid_ratio + test_ratio - 1.0) < tolerance, "The sum of the split ratios must be 1.0"

# Get list of all files (assuming each image has a corresponding label file with the same name but different extension)
image_files = [f for f in os.listdir(source_images_dir) if f.endswith('.jpg')]
random.shuffle(image_files)

# Calculate the split indices
total_files = len(image_files)
train_end = int(train_ratio * total_files)
valid_end = train_end + int(valid_ratio * total_files)

# Split the files
train_files = image_files[:train_end]
valid_files = image_files[train_end:valid_end]
test_files = image_files[valid_end:]

# Function to create directories if they don't exist and move files
def move_files(file_list, source_images_dir, source_labels_dir, dest_images_dir, dest_labels_dir):
    os.makedirs(dest_images_dir, exist_ok=True)
    os.makedirs(dest_labels_dir, exist_ok=True)
    for file_name in file_list:
        image_src_path = os.path.join(source_images_dir, file_name)
        label_src_path = os.path.join(source_labels_dir, file_name.replace('.jpg', '.txt'))
        image_dest_path = os.path.join(dest_images_dir, file_name)
        label_dest_path = os.path.join(dest_labels_dir, file_name.replace('.jpg', '.txt'))
        shutil.copyfile(image_src_path, image_dest_path)
        shutil.copyfile(label_src_path, label_dest_path)

# Define destination paths
train_images_dir = os.path.join(base_dest_dir, 'train', 'images')
train_labels_dir = os.path.join(base_dest_dir, 'train', 'labels')
valid_images_dir = os.path.join(base_dest_dir, 'valid', 'images')
valid_labels_dir = os.path.join(base_dest_dir, 'valid', 'labels')
test_images_dir = os.path.join(base_dest_dir, 'test', 'images')
test_labels_dir = os.path.join(base_dest_dir, 'test', 'labels')

# Move the files
move_files(train_files, source_images_dir, source_labels_dir, train_images_dir, train_labels_dir)
move_files(valid_files, source_images_dir, source_labels_dir, valid_images_dir, valid_labels_dir)
move_files(test_files, source_images_dir, source_labels_dir, test_images_dir, test_labels_dir)

print("Dataset split completed successfully.")

```

Dataset split completed successfully.

```

import shutil
from google.colab import files

# Define the path to the folder you want to download
folder_to_download = '/content/yolov5/runs/train/exp2/weights/best-fp16.tflite'

# Define the path for the zip file
zip_filename = '/content/yolov5/runs/train/exp2/weights/best-fp16.tflite.zip'

```

```
# Zip the folder
shutil.make_archive(zip_filename.replace('.zip', ''), 'zip', folder_to_download)

# Download the zip file
files.download(zip_filename)
```

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key=[REDACTED])
project = rf.workspace("nivin-a").project("fruits-hnsor")
version = project.version(3)
dataset = version.download("yolov5")
```

```
Collecting roboflow
  Downloading roboflow-1.1.36-py3-none-any.whl (76 kB)
----- 76.7/76.7 kB 3.3 MB/s eta 0:00:00
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Collecting chardet==4.0.0 (from roboflow)
  Downloading chardet-4.0.0-py2.py3-none-any.whl (178 kB)
----- 178.7/178.7 kB 8.3 MB/s eta 0:00:00
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cyclur in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Collecting python-dotenv (from roboflow)
  Downloading python_dotenv-1.0.1-py3-none-any.whl (19 kB)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Collecting requests-toolbelt (from roboflow)
  Downloading requests_toolbelt-1.0.0-py2.py3-none-any.whl (54 kB)
----- 54.5/54.5 kB 8.6 MB/s eta 0:00:00
Collecting filetype (from roboflow)
  Downloading filetype-1.2.0-py2.py3-none-any.whl (19 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
Installing collected packages: filetype, python-dotenv, chardet, requests-toolbelt, roboflow
  Attempting uninstall: chardet
    Found existing installation: chardet 5.2.0
    Uninstalling chardet-5.2.0:
      Successfully uninstalled chardet-5.2.0
Successfully installed chardet-4.0.0 filetype-1.2.0 python-dotenv-1.0.1 requests-toolbelt-1.0.0 roboflow-1.1.36
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in fruits-3 to yolov5pytorch:: 100%|██████████████████| 35576/35576 [00:01<00:00, 34934.15it/s]

Extracting Dataset Version Zip to fruits-3 in yolov5pytorch:: 100%|██████████████████| 4386/4386 [00:00<00:00, 9616.73it/s]
```

```
import os

# Define the path for the new folder
new_folder_path = '/content/anno_images1'

# Create the folder
os.makedirs(new_folder_path, exist_ok=True)
```

```
import os

# Define the path for the new folder
new_folder_path = '/content/images3'

# Create the folder
os.makedirs(new_folder_path, exist_ok=True)
```

```
import shutil

# Define the path to the folder you want to delete
folder_to_delete = '/content/balanced_dataset'

# Delete the folder
```

```
shutil.rmtree(folder_to_delete)
```

```
import shutil

# Define the path to the folder you want to delete
folder_to_delete = '/content/images10'

# Delete the folder
shutil.rmtree(folder_to_delete)
```

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key= [REDACTED] )
project = rf.workspace("applesbob").project("applebapples")
version = project.version(2)
dataset = version.download("yolov5")
```

Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.36)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Requirement already satisfied: chardet==4.0.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.0.0)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cyclerv in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in AppleBapples-2 to yolov5pytorch:: 100%[████████████████████] 100935/100935 [00:02<00:00, 42741.28it/s]

Extracting Dataset Version Zip to AppleBapples-2 in yolov5pytorch:: 100%[████████████████████] 9630/9630 [00:02<00:00, 3747.13it/s]

```
import os
import shutil

# Define the classes you want to keep and their new labels
class_mapping = {0: 1, 1: 3}

# Define paths
source_images_dir = '/content/AppleBapples-2/test/images'
source_labels_dir = '/content/AppleBapples-2/test/labels'
dest_images_dir = '/content/images3'
dest_labels_dir = '/content/anno_images3'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                old_class_id = int(line.split()[0])
                if old_class_id in class_mapping:
                    new_class_id = class_mapping[old_class_id]
                    new_line = line.replace(str(old_class_id), str(new_class_id), 1)
                    dst_file.write(new_line)

# Check if the filtered and relabeled label file is empty
if os.path.getsize(dest_label_path) == 0:
```



```

        os.remove(dest_label_path)
    else:
        # Copy the corresponding image if the label file is not empty
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define paths for the first dataset
source_images_dir1 = '/content/images2'
source_labels_dir1 = '/content/anno_images2'

# Define paths for the second dataset
source_images_dir2 = '/content/images3'
source_labels_dir2 = '/content/anno_images3'

# Define the destination paths
dest_images_dir = '/content/combined_dataset/images'
dest_labels_dir = '/content/combined_dataset/labels'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def copy_annotations_and_images(source_images_dir, source_labels_dir, dest_images_dir, dest_labels_dir):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

# Copy files from the first dataset
copy_annotations_and_images(source_images_dir1, source_labels_dir1, dest_images_dir, dest_labels_dir)

# Copy files from the second dataset
copy_annotations_and_images(source_images_dir2, source_labels_dir2, dest_images_dir, dest_labels_dir)

print("Datasets combined successfully.")

```

Datasets combined successfully.

```

import os
from collections import defaultdict

# Define the path to the directory containing annotation files
annotations_dir = '/content/balanced_dataset/labels'

# Initialize a dictionary to hold the class counts
class_counts = defaultdict(int)

# Iterate through each annotation file in the directory
for annotation_file in os.listdir(annotations_dir):
    if annotation_file.endswith('.txt'):
        # Construct the full path to the annotation file
        annotation_path = os.path.join(annotations_dir, annotation_file)

        # Open and read the annotation file
        with open(annotation_path, 'r') as file:
            lines = file.readlines()

            # Update class counts based on the first number of each line
            for line in lines:
                class_id = int(line.split()[0])

```

```
class_counts[class_id] += 1

# Display the class counts
print("Class Distribution in Dataset:")
for class_id, count in sorted(class_counts.items()):
    print(f"Class {class_id}: {count} instances")
```

```
Class Distribution in Dataset:
Class 0: 3853 instances
Class 1: 4684 instances
Class 2: 3504 instances
Class 3: 3639 instances
Class 4: 4520 instances
Class 5: 3496 instances
```

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key=[REDACTED])
project = rf.workspace("jawaharlal-nehru-university-utour").project("fruits-detection-and-quality-analysis")
version = project.version(9)
dataset = version.download("yolov5")
```

```
Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.36)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Requirement already satisfied: chardet==4.0.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.0.0)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Fruits-Detection-and-Quality-Analysis-9 to yolov5pytorch:: 100%|██████████████████| 149605/149605 [
Extracting Dataset Version Zip to Fruits-Detection-and-Quality-Analysis-9 in yolov5pytorch:: 100%|██████████████████| 10442/10442 [00:
```

```
import os
import shutil

# Define the classes you want to keep
classes_to_keep = {0, 1, 4, 5, 16, 17} # Example: keeping class 0, 2, and 3

# Define paths
source_images_dir = '/content/Fruits-Detection-and-Quality-Analysis-9/test/images'
source_labels_dir = '/content/Fruits-Detection-and-Quality-Analysis-9/test/labels'
dest_images_dir = '/content/images4'
dest_labels_dir = '/content/anno_images4'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep):
    valid_images = set() # To store the names of valid images

    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

            with open(dest_label_path, 'w') as dst_file:
                for line in lines:
                    class_id = int(line.split()[0])
```

```

        if class_id in classes_to_keep:
            dst_file.write(line)

    # Check if the filtered label file is empty
    if os.path.getsize(dest_label_path) == 0:
        os.remove(dest_label_path)
    else:
        # Add the corresponding image to the valid images set if the label file is not empty
        image_filename = filename.replace('.txt', '.jpg')
        valid_images.add(image_filename)

    # Copy valid images
    for image_filename in valid_images:
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define the classes you want to keep and their new labels
class_mapping = {0: 1, 1: 3, 4: 0, 5: 4, 16: 2, 17: 5}

# Define paths
source_images_dir = '/content/images4'
source_labels_dir = '/content/anno_images4'
dest_images_dir = '/content/images5'
dest_labels_dir = '/content/anno_images5'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                old_class_id = int(line.split()[0])
                if old_class_id in class_mapping:
                    new_class_id = class_mapping[old_class_id]
                    new_line = line.replace(str(old_class_id), str(new_class_id), 1)
                    dst_file.write(new_line)

    # Check if the filtered and relabeled label file is empty
    if os.path.getsize(dest_label_path) == 0:
        os.remove(dest_label_path)
    else:
        # Copy the corresponding image if the label file is not empty
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define paths for the new dataset
new_source_images_dir = '/content/images5'
new_source_labels_dir = '/content/anno_images5'

# Define paths for the existing combined dataset

```

```
combined_images_dir = '/content/combined_dataset/images'
combined_labels_dir = '/content/combined_dataset/labels'

# Function to copy files from new dataset to combined dataset
def add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir):
    for filename in os.listdir(new_source_labels_dir):
        # Define the source and destination paths for labels
        source_label_path = os.path.join(new_source_labels_dir, filename)
        dest_label_path = os.path.join(combined_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(new_source_images_dir, image_filename)
        dest_image_path = os.path.join(combined_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

# Add the new dataset to the combined dataset
add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir)

print("New dataset added to the combined dataset successfully.")
```

New dataset added to the combined dataset successfully.

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key=[REDACTED])
project = rf.workspace("ahmd5448").project("rotten-banana-wjjw1")
version = project.version(3)
dataset = version.download("yolov5")
```

```
Collecting roboflow
  Downloading roboflow-1.1.36-py3-none-any.whl (76 kB)
----- 76.7/76.7 kB 601.2 kB/s eta 0:00:00
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Collecting chardet==4.0.0 (from roboflow)
  Downloading chardet-4.0.0-py2.py3-none-any.whl (178 kB)
----- 178.7/178.7 kB 3.2 MB/s eta 0:00:00
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cyclur in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Collecting python-dotenv (from roboflow)
  Downloading python-dotenv-1.0.1-py3-none-any.whl (19 kB)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Collecting requests-toolbelt (from roboflow)
  Downloading requests-toolbelt-1.0.0-py2.py3-none-any.whl (54 kB)
----- 54.5/54.5 kB 3.3 MB/s eta 0:00:00
Collecting filetype (from roboflow)
  Downloading filetype-1.2.0-py2.py3-none-any.whl (19 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
Installing collected packages: filetype, python-dotenv, chardet, requests-toolbelt, roboflow
  Attempting uninstall: chardet
    Found existing installation: chardet 5.2.0
    Uninstalling chardet-5.2.0:
      Successfully uninstalled chardet-5.2.0
Successfully installed chardet-4.0.0 filetype-1.2.0 python-dotenv-1.0.1 requests-toolbelt-1.0.0 roboflow-1.1.36
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Rotten-Banana-3 to yolov5pytorch:: 100%|██████████████████| 80018/80018 [00:02<00:00, 39382.40it/s]

Extracting Dataset Version Zip to Rotten-Banana-3 in yolov5pytorch:: 100%|██████████████████| 2516/2516 [00:00<00:00, 2881.34it/s]
```

```
import os
import shutil

# Define the classes you want to keep and their new labels
class_mapping = {0: 0, 1: 4}
```

```

# Define paths
source_images_dir = '/content/Rotten-Banana-3/test/images'
source_labels_dir = '/content/Rotten-Banana-3/test/labels'
dest_images_dir = '/content/images6'
dest_labels_dir = '/content/anno_images6'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                old_class_id = int(line.split()[0])
                if old_class_id in class_mapping:
                    new_class_id = class_mapping[old_class_id]
                    new_line = line.replace(str(old_class_id), str(new_class_id), 1)
                    dst_file.write(new_line)

        # Check if the filtered and relabeled label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            # Copy the corresponding image if the label file is not empty
            image_filename = filename.replace('.txt', '.jpg')
            source_image_path = os.path.join(source_images_dir, image_filename)
            dest_image_path = os.path.join(dest_images_dir, image_filename)
            shutil.copyfile(source_image_path, dest_image_path)

    filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define paths for the new dataset
new_source_images_dir = '/content/images6'
new_source_labels_dir = '/content/anno_images6'

# Define paths for the existing combined dataset
combined_images_dir = '/content/combined_dataset/images'
combined_labels_dir = '/content/combined_dataset/labels'

# Function to copy files from new dataset to combined dataset
def add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir):
    for filename in os.listdir(new_source_labels_dir):
        # Define the source and destination paths for labels
        source_label_path = os.path.join(new_source_labels_dir, filename)
        dest_label_path = os.path.join(combined_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(new_source_images_dir, image_filename)
        dest_image_path = os.path.join(combined_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

    # Add the new dataset to the combined dataset
    add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir)

print("New dataset added to the combined dataset successfully.")

```

New dataset added to the combined dataset successfully.

```
!pip install roboflow
```

```
from roboflow import Roboflow
```

```

rf = Roboflow(api_key=[REDACTED])
project = rf.workspace("maryam-saidbauwain-pxjil").project("banana-hwclp")
version = project.version(1)
dataset = version.download("yolov5")

```

```

Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.36)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Requirement already satisfied: chardet==4.0.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.0.0)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in banana-1 to yolov5pytorch:: 100%|██████████████████| 980067/980067 [00:27<00:00, 35631.86it/s]

Extracting Dataset Version Zip to banana-1 in yolov5pytorch:: 100%|██████████████████| 19912/19912 [00:11<00:00, 1665.78it/s]

```

```

import os
import shutil

# Define the classes you want to keep
classes_to_keep = {0, 4} # Example: keeping class 0, 2, and 3

# Define paths
source_images_dir = '/content/banana-1/valid/images'
source_labels_dir = '/content/banana-1/valid/labels'
dest_images_dir = '/content/images7'
dest_labels_dir = '/content/anno_images7'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep):
    valid_images = set() # To store the names of valid images

    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                class_id = int(line.split()[0])
                if class_id in classes_to_keep:
                    dst_file.write(line)

        # Check if the filtered label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            # Add the corresponding image to the valid images set if the label file is not empty
            image_filename = filename.replace('.txt', '.jpg')
            valid_images.add(image_filename)

    # Copy valid images
    for image_filename in valid_images:
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep)

```

```
print("Filtering and relabeling completed successfully.")
```

Filtering and relabeling completed successfully.

```
import os
import shutil

# Define paths for the new dataset
new_source_images_dir = '/content/images7'
new_source_labels_dir = '/content/anno_images7'

# Define paths for the existing combined dataset
combined_images_dir = '/content/combined_dataset/images'
combined_labels_dir = '/content/combined_dataset/labels'

# Function to copy files from new dataset to combined dataset
def add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir):
    for filename in os.listdir(new_source_labels_dir):
        # Define the source and destination paths for labels
        source_label_path = os.path.join(new_source_labels_dir, filename)
        dest_label_path = os.path.join(combined_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(new_source_images_dir, image_filename)
        dest_image_path = os.path.join(combined_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

# Add the new dataset to the combined dataset
add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir)

print("New dataset added to the combined dataset successfully.")
```

New dataset added to the combined dataset successfully.

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key= [REDACTED] )
project = rf.workspace("object-detection-fmrts").project("fresh-rotten-oranges")
version = project.version(3)
dataset = version.download("yolov5")
```

Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.36)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Requirement already satisfied: chardet==4.0.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.0.0)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Fresh-Rotten-Oranges-3 to yolov5pytorch:: 100%[████████████████████] 71193/71193 [00:02<00:00, 31358.

Extracting Dataset Version Zip to Fresh-Rotten-Oranges-3 in yolov5pytorch:: 100%[████████████████████] 3144/3144 [00:01<00:00, 2618.75it

```
import os
import shutil

# Define the classes you want to keep and their new labels
class_mapping = {0: 2, 1: 5}
```

```

# Define paths
source_images_dir = '/content/Fresh-Rotten-Oranges-3/valid/images'
source_labels_dir = '/content/Fresh-Rotten-Oranges-3/valid/labels'
dest_images_dir = '/content/images8'
dest_labels_dir = '/content/anno_images8'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                old_class_id = int(line.split()[0])
                if old_class_id in class_mapping:
                    new_class_id = class_mapping[old_class_id]
                    new_line = line.replace(str(old_class_id), str(new_class_id), 1)
                    dst_file.write(new_line)

        # Check if the filtered and relabeled label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            # Copy the corresponding image if the label file is not empty
            image_filename = filename.replace('.txt', '.jpg')
            source_image_path = os.path.join(source_images_dir, image_filename)
            dest_image_path = os.path.join(dest_images_dir, image_filename)
            shutil.copyfile(source_image_path, dest_image_path)

    filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define paths for the new dataset
new_source_images_dir = '/content/images8'
new_source_labels_dir = '/content/anno_images8'

# Define paths for the existing combined dataset
combined_images_dir = '/content/combined_dataset/images'
combined_labels_dir = '/content/combined_dataset/labels'

# Function to copy files from new dataset to combined dataset
def add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir):
    for filename in os.listdir(new_source_labels_dir):
        # Define the source and destination paths for labels
        source_label_path = os.path.join(new_source_labels_dir, filename)
        dest_label_path = os.path.join(combined_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(new_source_images_dir, image_filename)
        dest_image_path = os.path.join(combined_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

    # Add the new dataset to the combined dataset
    add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir)

print("New dataset added to the combined dataset successfully.")

```

New dataset added to the combined dataset successfully.

```
!pip install roboflow
```

```
from roboflow import Roboflow
```



```

rf = Roboflow(api_key=[REDACTED])
project = rf.workspace("neha-chandekar-yxsnl").project("fresh-rotten-orange-classifier")
version = project.version(2)
dataset = version.download("yolov5")

```

```

Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.36)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Requirement already satisfied: chardet==4.0.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.0.0)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Fresh-Rotten-Orange-Classfier-2 to yolov5pytorch:: 100%|██████████████████| 65924/65924 [00:01<00:
Extracting Dataset Version Zip to Fresh-Rotten-Orange-Classfier-2 in yolov5pytorch:: 100%|██████████████████| 7716/7716 [00:02<00:00,

```

```

import os
import shutil

# Define the classes you want to keep and their new labels
class_mapping = {0: 2, 1: 5}

# Define paths
source_images_dir = '/content/Fresh-Rotten-Orange-Classfier-2/valid/images'
source_labels_dir = '/content/Fresh-Rotten-Orange-Classfier-2/valid/labels'
dest_images_dir = '/content/images9'
dest_labels_dir = '/content/anno_images9'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping):
    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                old_class_id = int(line.split()[0])
                if old_class_id in class_mapping:
                    new_class_id = class_mapping[old_class_id]
                    new_line = line.replace(str(old_class_id), str(new_class_id), 1)
                    dst_file.write(new_line)

        # Check if the filtered and relabeled label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            # Copy the corresponding image if the label file is not empty
            image_filename = filename.replace('.txt', '.jpg')
            source_image_path = os.path.join(source_images_dir, image_filename)
            dest_image_path = os.path.join(dest_images_dir, image_filename)
            shutil.copyfile(source_image_path, dest_image_path)

    filter_and_relabel_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, class_mapping)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```
import os
import shutil

# Define paths for the new dataset
new_source_images_dir = '/content/images9'
new_source_labels_dir = '/content/anno_images9'

# Define paths for the existing combined dataset
combined_images_dir = '/content/combined_dataset/images'
combined_labels_dir = '/content/combined_dataset/labels'

# Function to copy files from new dataset to combined dataset
def add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir):
    for filename in os.listdir(new_source_labels_dir):
        # Define the source and destination paths for labels
        source_label_path = os.path.join(new_source_labels_dir, filename)
        dest_label_path = os.path.join(combined_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(new_source_images_dir, image_filename)
        dest_image_path = os.path.join(combined_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

# Add the new dataset to the combined dataset
add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir)

print("New dataset added to the combined dataset successfully.")
```

New dataset added to the combined dataset successfully.

```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key=[REDACTED])
project = rf.workspace("fruit-ripening").project("banana-ripening-process")
version = project.version(2)
dataset = version.download("yolov5")
```

Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.36)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.7.4)
Requirement already satisfied: chardet==4.0.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.0.0)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.25.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (9.4.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.0.7)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.4)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.1)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.53.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.3.2)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Banana-Ripening-Process-2 to yolov5pytorch:: 100%
Extracting Dataset Version Zip to Banana-Ripening-Process-2 in yolov5pytorch:: 100%

```
import os
import shutil

# Define the classes you want to keep
classes_to_keep = {0} # Example: keeping class 0, 2, and 3

# Define paths
```

```

source_images_dir = '/content/Banana-Ripening-Process-2/valid/images'
source_labels_dir = '/content/Banana-Ripening-Process-2/valid/labels'
dest_images_dir = '/content/images10'
dest_labels_dir = '/content/anno_images10'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

def filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep):
    valid_images = set() # To store the names of valid images

    for filename in os.listdir(source_labels_dir):
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)

        with open(source_label_path, 'r') as src_file:
            lines = src_file.readlines()

        with open(dest_label_path, 'w') as dst_file:
            for line in lines:
                class_id = int(line.split()[0])
                if class_id in classes_to_keep:
                    dst_file.write(line)

        # Check if the filtered label file is empty
        if os.path.getsize(dest_label_path) == 0:
            os.remove(dest_label_path)
        else:
            # Add the corresponding image to the valid images set if the label file is not empty
            image_filename = filename.replace('.txt', '.jpg')
            valid_images.add(image_filename)

    # Copy valid images
    for image_filename in valid_images:
        source_image_path = os.path.join(source_images_dir, image_filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

filter_annotations(source_labels_dir, dest_labels_dir, source_images_dir, dest_images_dir, classes_to_keep)

print("Filtering and relabeling completed successfully.")

```

Filtering and relabeling completed successfully.

```

import os
import shutil

# Define paths for the new dataset
new_source_images_dir = '/content/images10'
new_source_labels_dir = '/content/anno_images10'

# Define paths for the existing combined dataset
combined_images_dir = '/content/combined_dataset/images'
combined_labels_dir = '/content/combined_dataset/labels'

# Function to copy files from new dataset to combined dataset
def add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir):
    for filename in os.listdir(new_source_labels_dir):
        # Define the source and destination paths for labels
        source_label_path = os.path.join(new_source_labels_dir, filename)
        dest_label_path = os.path.join(combined_labels_dir, filename)

        # Copy the label file
        shutil.copyfile(source_label_path, dest_label_path)

        # Copy the corresponding image file
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(new_source_images_dir, image_filename)
        dest_image_path = os.path.join(combined_images_dir, image_filename)
        shutil.copyfile(source_image_path, dest_image_path)

    # Add the new dataset to the combined dataset
    add_to_combined_dataset(new_source_images_dir, new_source_labels_dir, combined_images_dir, combined_labels_dir)

print("New dataset added to the combined dataset successfully.")

```

New dataset added to the combined dataset successfully.

```

# Install PyDrive
!pip install -U -q PyDrive

# Import necessary libraries
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials
import os
import shutil
from google.colab import files

# Authenticate and create the PyDrive client
auth.authenticate_user()
gauth = GoogleAuth()
gauth.credentials = GoogleCredentials.get_application_default()
drive = GoogleDrive(gauth)

# Function to download a folder from Google Drive
def download_folder(folder_id, destination):
    # Create a GoogleDriveFile object for the folder
    folder = drive.CreateFile({'id': folder_id})
    folder.FetchMetadata(fetch_all=True)
    folder_name = folder['title']

    # Create the destination directory
    os.makedirs(destination, exist_ok=True)

    # List all files and subfolders in the folder
    file_list = drive.ListFile({'q': f"'{folder_id}' in parents and trashed=false"}).GetList()

    for file in file_list:
        if file['mimeType'] == 'application/vnd.google-apps.folder':
            # If the file is a folder, recurse into
            print(f"Found folder: {file['title']}")
            subfolder_id = file['id']
            subfolder_name = file['title']
            subfolder_path = os.path.join(destination, subfolder_name)
            download_folder(subfolder_id, subfolder_path)
        else:
            # If the file is not a folder, download it
            file_path = os.path.join(destination, file['title'])
            file.GetContentFile(file_path)

# Download the folder
folder_id = '1Cyic2U4zlopvEI8pvMeLCV_df1kEfui1' # Replace with your actual folder ID
destination = '/content/combined_dataset1'
download_folder(folder_id, destination)

# Zip the downloaded folder
shutil.make_archive(destination, 'zip', destination)

# Download the zip file
files.download(destination + '.zip')

# Clean up (optional)
shutil.rmtree(destination) # Remove the unzipped folder
os.remove(destination + '.zip') # Remove the zip file after download (optional)

```

Found folder: labels

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-9-184e80a55b73> in <cell line: 48>()
    46 folder_id = '1Cyic2U4zlopvEI8pvMeLCV_df1kEfui1' # Replace with your actual folder ID
    47 destination = '/content/combined_dataset1'
--> 48 download_folder(folder_id, destination)
    49
    50 # Zip the downloaded folder

<ipython-input-9-184e80a55b73> in download_folder(folder_id, destination)
    37         subfolder_name = file['title']
    38         subfolder_path = os.path.join(destination, subfolder_name)
--> 39         download_folder(subfolder_id, subfolder_path)
    40     else:
    41         # If the file is not a folder, download it

<ipython-input-9-184e80a55b73> in download_folder(folder_id, destination)
    41         # If the file is not a folder, download it
    42         file_path = os.path.join(destination, file['title'])
    43         file.GetContentFile(file_path)

```

```

import os
import shutil

```

```

dataset_path = [REDACTED] [REDACTED] combined_dataset'

# Now you can use the dataset path to load your data
# Example: List files in the dataset directory
import os

for file_name in os.listdir(dataset_path):
    print(file_name)
# Define the source and destination paths
source_images_dir = os.path.join(dataset_path, 'images')
source_labels_dir = os.path.join(dataset_path, 'labels')
dest_images_dir = '/content/images'
dest_labels_dir = '/content/labels'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

# Function to copy files from Google Drive to Colab environment
def copy_files(source_dir, dest_dir):
    for file_name in os.listdir(source_dir):
        source_path = os.path.join(source_dir, file_name)
        dest_path = os.path.join(dest_dir, file_name)
        shutil.copy(source_path, dest_path)

# Copy image and label files
copy_files(source_images_dir, dest_images_dir)
copy_files(source_labels_dir, dest_labels_dir)

print("Files copied successfully.")

```

```

images
labels

```

```

-----
OSError                                Traceback (most recent call last)
<ipython-input-2-cla657dd8b83> in <cell line: 31>()
    29
    30 # Copy image and label files
--> 31 copy_files(source_images_dir, dest_images_dir)
    32 copy_files(source_labels_dir, dest_labels_dir)
    33

<ipython-input-2-cla657dd8b83> in copy_files(source_dir, dest_dir)
    23 # Function to copy files from Google Drive to Colab environment
    24 def copy_files(source_dir, dest_dir):
--> 25     for file_name in os.listdir(source_dir):
    26         source_path = os.path.join(source_dir, file_name)
    27         dest_path = os.path.join(dest_dir, file_name)

OSError: [Errno 5] Input/output error: [REDACTED] [REDACTED] combined_dataset/images'

```

```

!pip install -U -q PyDrive

from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials
import zipfile

# Authenticate and create the PyDrive client.
auth.authenticate_user()
gauth = GoogleAuth()
gauth.credentials = GoogleCredentials.get_application_default()
drive = GoogleDrive(gauth)

def download_file(file_id, dest_path):
    downloaded = drive.CreateFile({'id': file_id})
    downloaded.GetContentFile(dest_path)
    print(f"Downloaded {dest_path}")

file_id = [REDACTED] # Replace with your actual file ID
dest_path = '/content/combined_dataset1.zip' # Replace with your desired destination file name

download_file(file_id, dest_path)

```

WARNING:root:pydrive is deprecated and no longer maintained. We recommend that you migrate your projects to pydrive2, the maintained fork. Downloaded /content/combined_dataset1.zip

```

import os
import random
import shutil

# Define paths
source_images_dir = '/images'
source_labels_dir = '/labels'
dest_images_dir = '/content/balanced_dataset/images'
dest_labels_dir = '/content/balanced_dataset/labels'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

# Define the target number of instances per class
target_count = 3500

# Initialize a dictionary to store the filenames by class
class_files = {0: [], 1: [], 2: [], 3: [], 4: [], 5: []}

# Read all label files and categorize images by class
for filename in os.listdir(source_labels_dir):
    source_label_path = os.path.join(source_labels_dir, filename)
    with open(source_label_path, 'r') as file:
        lines = file.readlines()
        for line in lines:
            class_id = int(line.split()[0])
            if class_id in class_files:
                class_files[class_id].append(filename)

# Sample files to balance the dataset
balanced_files = {cls: (files if len(files) <= target_count else random.sample(files, target_count))
                  for cls, files in class_files.items()}

# Function to copy files
def copy_files(file_list, source_images_dir, source_labels_dir, dest_images_dir, dest_labels_dir):
    for filename in file_list:
        image_filename = filename.replace('.txt', '.jpg')
        source_image_path = os.path.join(source_images_dir, image_filename)
        source_label_path = os.path.join(source_labels_dir, filename)
        dest_image_path = os.path.join(dest_images_dir, image_filename)
        dest_label_path = os.path.join(dest_labels_dir, filename)
        shutil.copyfile(source_image_path, dest_image_path)
        shutil.copyfile(source_label_path, dest_label_path)

# Copy the balanced files to the new directory
for cls, files in balanced_files.items():
    copy_files(files, source_images_dir, source_labels_dir, dest_images_dir, dest_labels_dir)

print("Dataset balancing completed successfully.")

```

Dataset balancing completed successfully.

```

import os
import shutil
import random
from collections import defaultdict

# Define paths
source_images_dir = '/images'
source_labels_dir = '/labels'
dest_images_dir = '/content/balanced_dataset/images'
dest_labels_dir = '/content/balanced_dataset/labels'

# Create destination directories if they don't exist
os.makedirs(dest_images_dir, exist_ok=True)
os.makedirs(dest_labels_dir, exist_ok=True)

# Define the target number of instances per class
target_per_class = 3500

# Count instances per class
def count_class_instances(labels_dir):
    class_instances = defaultdict(list)
    for label_file in os.listdir(labels_dir):
        with open(os.path.join(labels_dir, label_file), 'r') as f:
            lines = f.readlines()
            for line in lines:
                class_id = int(line.split()[0])

```

```

        class_instances[class_id].append(label_file)

    return class_instances

class_instances = count_class_instances(source_labels_dir)

# Sample instances and copy files
def sample_and_copy_files(class_instances, source_images_dir, source_labels_dir, dest_images_dir, dest_labels_dir, target_per_class):
    for class_id, files in class_instances.items():
        # If more than target instances, sample target_per_class
        if len(files) > target_per_class:
            sampled_files = random.sample(files, target_per_class)
        else:
            sampled_files = files

        for label_file in sampled_files:
            # Copy the label file
            source_label_path = os.path.join(source_labels_dir, label_file)
            dest_label_path = os.path.join(dest_labels_dir, label_file)
            shutil.copyfile(source_label_path, dest_label_path)

            # Copy the corresponding image file
            image_file = label_file.replace('.txt', '.jpg')
            source_image_path = os.path.join(source_images_dir, image_file)
            dest_image_path = os.path.join(dest_images_dir, image_file)
            shutil.copyfile(source_image_path, dest_image_path)

sample_and_copy_files(class_instances, source_images_dir, source_labels_dir, dest_images_dir, dest_labels_dir, target_per_class)

print("Dataset balancing completed successfully.")

```

Dataset balancing completed successfully.

```

import os

# Define paths
image_dir = '/content/balanced_dataset/images' # Update this with the path to your images directory
annotation_dir = '/content/balanced_dataset/labels' # Update this with the path to your annotations directory

# Get list of image files and annotation files
image_files = [f for f in os.listdir(image_dir) if f.endswith('.jpg')] # Change the extension if your images are not
annotation_files = [f for f in os.listdir(annotation_dir) if f.endswith('.txt')] # Change the extension if your annotations are not

# Check for equality
num_images = len(image_files)
num_annotations = len(annotation_files)

print(f"Number of images: {num_images}")
print(f"Number of annotation files: {num_annotations}")

if num_images == num_annotations:
    print("The number of images and annotation files are equal.")
else:
    print("The number of images and annotation files are not equal.")

# Optionally, list missing or extra files
image_basenamees = {os.path.splitext(f)[0] for f in image_files}
annotation_basenamees = {os.path.splitext(f)[0] for f in annotation_files}

```